

E-Commerce Sales Database

Submitted By:

Kashish Goel

College:

DIT University

Internship:

Celebal Technologies

Submitted To:

Celebal Technologies

Date:

28 June 2026

Objective-

The objective of this assignment is to analyze the ShopEase database using SQL. The tasks include retrieving data, applying filters, performing aggregations, using joins, implementing CASE statements, understanding constraints and indexes, and executing transactions to generate meaningful business insights.

Database Schema

The ShopEase database consists of the following four tables:

- Customers
- Products
- Orders
- Order_Items

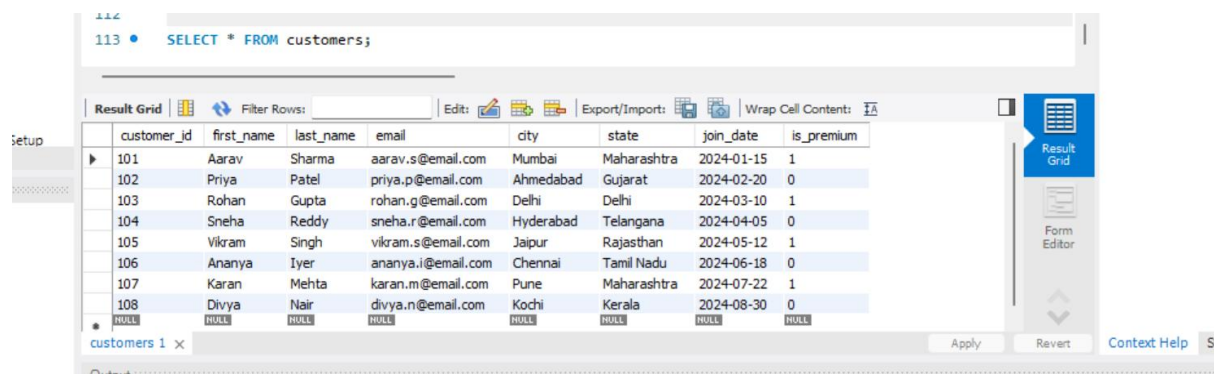
Section A – SQL Basics

Q1. Display all columns and rows from the customers table.

SQL Query-

```
SELECT * FROM customers;
```

Output



customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Brief Insight

This query displays all customer records available in the customers table.

Q2. Retrieve only the first_name, last_name, and city of all customers.

SQL Query

```
SELECT first_name, last_name, city  
FROM customers;
```

Output

```
115 • SELECT first_name, last_name, city
116 FROM customers;
```

	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Brief Insight

This query retrieves only the required customer details.

Q3. List all unique categories available in the products table.

SQL Query

```
SELECT DISTINCT category
FROM products;
```

Output

```
118 • SELECT DISTINCT category
119 FROM products;
```

	category
▶	Clothing
	Electronics
	Home

Brief Insight

The query displays all unique product categories available in the database.

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be UNIQUE and NOT NULL.

Answer

Table	Primary Key
Customers	customer_id
Products	product_id
Orders	order_id
Order_Items	item_id

A Primary Key uniquely identifies each record in a table. It must always be **UNIQUE** and **NOT NULL**, ensuring data integrity and preventing duplicate records.

Q5. What constraints are applied to the email column?

SQL Query

```
INSERT INTO customers
(customer_id, first_name, last_name, email, city, state, join_date,
is_premium)
VALUES
(109, 'Rahul', 'Kumar', 'aarav.s@email.com', 'Delhi', 'Delhi', '2024-09-
01', FALSE);
```

Output

✓	21	23:37:04	SELECT DISTINCT category FROM products LIMIT 0, 1000	3 row(s) returned	0.000 sec / 0.0
✗	22	23:38:08	INSERT INTO customers (customer_id, first_name, last_name, email, city, state, join... Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'		0.016 sec

Brief Insight

The **UNIQUE** constraint on the email column prevents duplicate email addresses from being inserted.

Q6. Insert a product with a negative price.

SQL Query

```
INSERT INTO products
(product_id, product_name, category, brand, unit_price, stock_qty)
VALUES
(209, 'Test Product', 'Electronics', 'TestBrand', -50, 10);
```

Output

24 23:39:58 INSERT INTO products (product_id, product_name, category, brand, unit_price, st... Error Code: 3819. Check constraint 'products_chk_1' is violated. 0.000 sec

Brief Insight

The CHECK constraint prevents products from having a negative unit price.

Q7. Retrieve all orders with status = 'Delivered'.

SQL Query

```
SELECT *
FROM orders
WHERE status = 'Delivered';
```

Output

133 • SELECT *
134 FROM orders
135 WHERE status = 'Delivered';

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Brief Insight

The query displays all orders whose status is marked as **Delivered**.

Section B – Filtering & Optimization

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

SQL Query

```
SELECT *  
FROM products  
WHERE category = 'Electronics'  
AND unit_price > 2000;
```

Output

☐ (Paste Screenshot)

Brief Insight

This query retrieves electronic products priced above ₹2000, helping identify premium products in the electronics category.

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

SQL Query

```
SELECT *  
FROM customers  
WHERE state = 'Maharashtra'  
AND YEAR(join_date) = 2024;  
Output-
```

```
133 • SELECT *
134 FROM orders
135 WHERE status = 'Delivered';
```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
*	NULL	NULL	NULL	NULL	NULL

Brief Insight

This query lists customers from Maharashtra who joined ShopEase in 2024.

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

SQL Query

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
AND status <> 'Cancelled';
```

Output

```
144 ● SELECT *
145     FROM orders
146     WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'
147     AND status <> 'Cancelled';
```

Result Grid		Filter Rows:		Edit:	Export/Imp	
	order_id	customer_id	order_date	status	total_amount	
▶	1004	101	2024-08-10	Delivered	3499.00	
	1006	105	2024-08-15	Delivered	5898.00	
	1007	106	2024-08-18	Pending	1299.00	
	1008	103	2024-08-20	Delivered	899.00	
	1009	107	2024-08-25	Shipped	6098.00	
✱	NULL	NULL	NULL	NULL	NULL	

Brief Insight

This query retrieves all active orders within the specified date range while excluding cancelled orders.

Q11. Explain the purpose of idx_orders_date.

Answer

The `idx_orders_date` index is created on the `order_date` column of the `orders` table. It improves query performance by allowing MySQL to quickly locate records based on dates instead of scanning the entire table.

Sample Query

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-20';
```

Output

```
149 • SELECT *
150 FROM orders
151 WHERE order_date BETWEEN '2024-08-10' AND '2024-08-20';
```

Result Grid						Filter Rows:	Edit:			Export/Imp
	order_id	customer_id	order_date	status	total_amount					
▶	1004	101	2024-08-10	Delivered	3499.00					
	1005	104	2024-08-12	Cancelled	2999.00					
	1006	105	2024-08-15	Delivered	5898.00					
	1007	106	2024-08-18	Pending	1299.00					
	1008	103	2024-08-20	Delivered	899.00					
•	NULL	NULL	NULL	NULL	NULL					

Brief Insight

The index speeds up date-based searches, especially when working with large datasets.

Q12. Rewrite the query to make it index-friendly (SARGable).

Answer

Using the `YEAR(join_date)` function prevents efficient index usage because MySQL must evaluate the function for every row. A date range allows the database to use the index effectively.

SQL Query

```
SELECT *
FROM customers
```

```
WHERE join_date >= '2024-01-01'  
AND join_date < '2025-01-01';
```

Output

```
153 • SELECT * FROM customers  
154 WHERE YEAR(join_date) = 2024;
```

Result Grid						
Filter Rows:						
Edit:    Export/Import						
	customer_id	first_name	last_name	email	city	state
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala
•	NULL	NULL	NULL	NULL	NULL	NULL

customers 11 ×

Brief Insight

Using a date range instead of the `YEAR()` function makes the query more efficient and allows the index to be utilized.

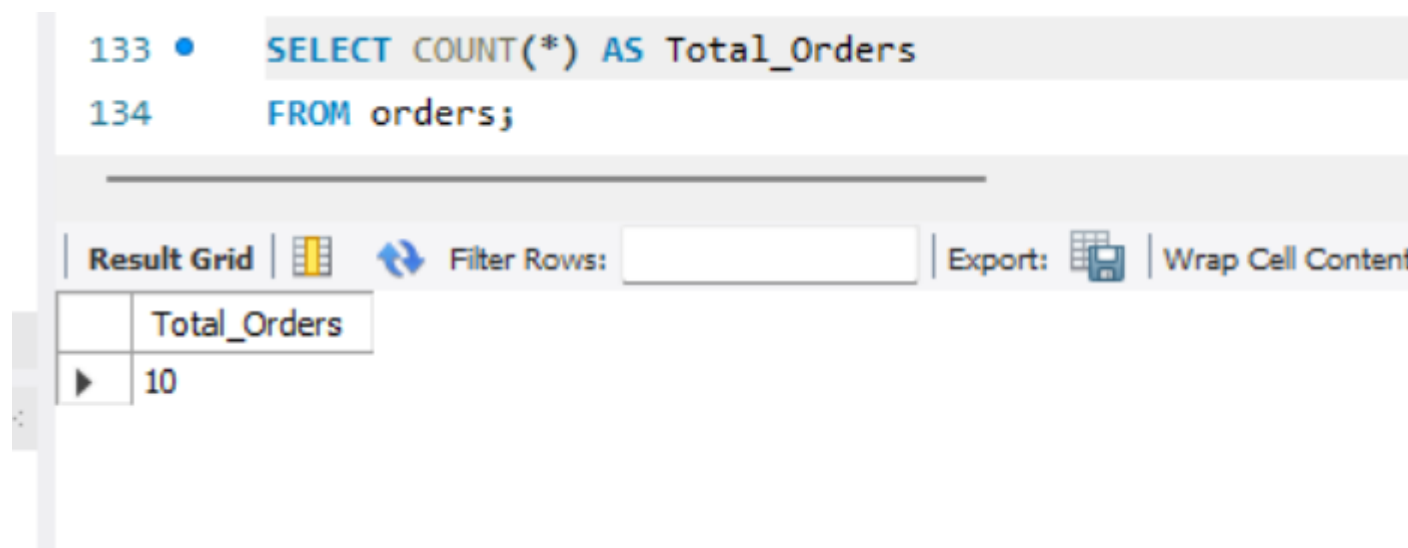
Section C – Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)-

Q13. Count the total number of orders in the orders table.

SQL Query

```
SELECT COUNT(*) AS Total_Orders  
FROM orders;
```

Output



The screenshot shows a SQL query editor with the following query:

```
133 • SELECT COUNT(*) AS Total_Orders  
134 FROM orders;
```

Below the query, there is a toolbar with options: Result Grid, Filter Rows, Export, and Wrap Cell Content. The output is displayed in a table with one row and one column:

Total_Orders
10

Brief Insight

This query counts the total number of orders present in the database.

Q14. Find the total revenue (SUM of total_amount) from all Delivered orders.

SQL Query

```
SELECT SUM(total_amount) AS Total_Revenue  
FROM orders  
WHERE status = 'Delivered';
```

Output

```
137 • SELECT SUM(total_amount) AS Total_Revenue
138 FROM orders
139 WHERE status = 'Delivered';
140
```

Result Grid |  Filter Rows: | Export:  | Wrap Cell Content:

	Total_Revenue
▶	17191.00

Brief Insight

This query calculates the total revenue generated from successfully delivered orders.

Q15. Calculate the average unit_price of products in each category.

SQL Query

```
SELECT category,
       AVG(unit_price) AS Average_Price
FROM products
GROUP BY category;
```

Output

```
142 • SELECT category,  
143     AVG(unit_price) AS Average_Price  
144     FROM products  
145     GROUP BY category;  
146
```

Result Grid		 Filter Rows:	Export: 	Wrap Cell Content: ☐
	category	Average_Price		
▶	Clothing	2699.000000		
	Electronics	2224.000000		
	Home	949.000000		

Brief Insight

This query calculates the average product price for each category.

Q16. For each order status, find the count of orders and total revenue.

SQL Query

```
SELECT status,  
       COUNT(*) AS Total_Orders,  
       SUM(total_amount) AS Total_Revenue  
FROM orders  
GROUP BY status  
ORDER BY Total_Revenue DESC;
```

Output

Result Grid				Filter Rows:	Export:	Wrap Cell Content:
	status	Total_Orders	Total_Revenue			
▶	Delivered	6	17191.00			
	Shipped	2	13596.00			
	Cancelled	1	2999.00			
	Pending	1	1299.00			

Brief Insight

This query summarizes the number of orders and total revenue for each order status.

Q17. Find the most expensive and cheapest product in each category.

SQL Query

```
SELECT category,
       MAX(unit_price) AS Highest_Price,
       MIN(unit_price) AS Lowest_Price
FROM products
GROUP BY category;
```

Output

Result Grid				Filter Rows:	Export:	Wrap Cell Content
	category	Highest_Price	Lowest_Price			
▶	Clothing	4599.00	799.00			
	Electronics	3499.00	899.00	899.00		
	Home	1299.00	599.00			

Brief Insight

This query identifies the highest-priced and lowest-priced products in each category.

Q18. List all product categories where the average unit_price is greater than ₹2000.

SQL Query

```
SELECT category,  
       AVG(unit_price) AS Average_Price  
FROM products  
GROUP BY category  
HAVING AVG(unit_price) > 2000;
```

Output

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	category	Average_Price			
▶	Clothing	2699.000000			
	Electronics	2224.000000			

Brief Insight

This query displays only those categories whose average product price exceeds ₹2000.

Section D – Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.

SQL Query

```
SELECT o.order_id,  
       o.order_date,  
       c.first_name,  
       c.last_name,  
       o.total_amount  
FROM orders o  
INNER JOIN customers c  
ON o.customer_id = c.customer_id;
```

Output

Result Grid		Filter Rows:		Export:	Wrap Cell Content:
	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Brief Insight

This query combines the **orders** and **customers** tables to display customer details for each order.

Q20. Using a LEFT JOIN, list all customers and their orders (if any).

SQL Query

```
SELECT c.customer_id,  
       c.first_name,  
       c.last_name,  
       o.order_id,  
       o.order_date,  
       o.total_amount  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id;
```

Output

Result Grid							Filter Rows:	Export:	Wrap Cell Content:
	customer_id	first_name	last_name	order_id	order_date	total_amount			
▶	101	Aarav	Sharma	1001	2024-08-01	4498.00			
	101	Aarav	Sharma	1004	2024-08-10	3499.00			
	102	Priya	Patel	1002	2024-08-03	799.00			
	103	Rohan	Gupta	1003	2024-08-05	7498.00			
	103	Rohan	Gupta	1008	2024-08-20	899.00			
	104	Sneha	Reddy	1005	2024-08-12	2999.00			
	105	Vikram	Singh	1006	2024-08-15	5898.00			
	106	Ananya	Iyer	1007	2024-08-18	1299.00			
	107	Karan	Mehta	1009	2024-08-25	6098.00			
	108	Divya	Nair	1010	2024-08-28	1598.00			

Brief Insight

This query displays all customers, including those who have not placed any orders. Customers without orders show **NULL** values for order details.

Q21. Display order_id, product_name, quantity, unit_price and discount_pct using JOINS.

SQL Query

```
SELECT o.order_id,
       p.product_name,
       oi.quantity,
       oi.unit_price,
       oi.discount_pct
FROM orders o
INNER JOIN order_items oi
ON o.order_id = oi.order_id
INNER JOIN products p
ON oi.product_id = p.product_id;
```

Output

Result Grid  Filter Rows: Export:  Wrap Cell Contents: ☐					
	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00
	1008	Laptop Stand	1	899.00	0.00
	1009	Cushion Covers (...)	2	599.00	15.00
	1010	Cushion Covers (...)	1	599.00	0.00

Brief Insight

This query joins three tables to display complete order item information, including product details, quantity, price, and discount.

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN. When would you use FULL OUTER JOIN?

Answer

- **LEFT JOIN** returns all rows from the left table and matching rows from the right table. If no match exists, NULL values are returned.
 - **RIGHT JOIN** returns all rows from the right table and matching rows from the left table. If no match exists, NULL values are returned.
 - **FULL OUTER JOIN** returns all matching and non-matching rows from both tables. It is useful when you want to include every record from both tables regardless of whether a match exists. (MySQL does not directly support FULL OUTER JOIN.)
-

Q23. Identify all Foreign Key relationships in the schema.

Answer

Foreign Key Relationships

- `orders.customer_id` → `customers.customer_id`
- `order_items.order_id` → `orders.order_id`
- `order_items.product_id` → `products.product_id`

If an order is inserted with a `customer_id` that does not exist (for example, `customer_id = 999`), MySQL will generate a **Foreign Key Constraint Error** and the record will not be inserted, maintaining referential integrity.

Q24. Classify products into price tiers using CASE.

SQL Query

```
SELECT product_name,  
       unit_price,  
       CASE  
         WHEN unit_price < 1000 THEN 'Budget'  
         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'  
         ELSE 'Premium'  
       END AS price_tier  
FROM products;
```

Output

Result Grid   Filter Rows: Export:  Wrap Cell Content: ☐			
	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Brief Insight

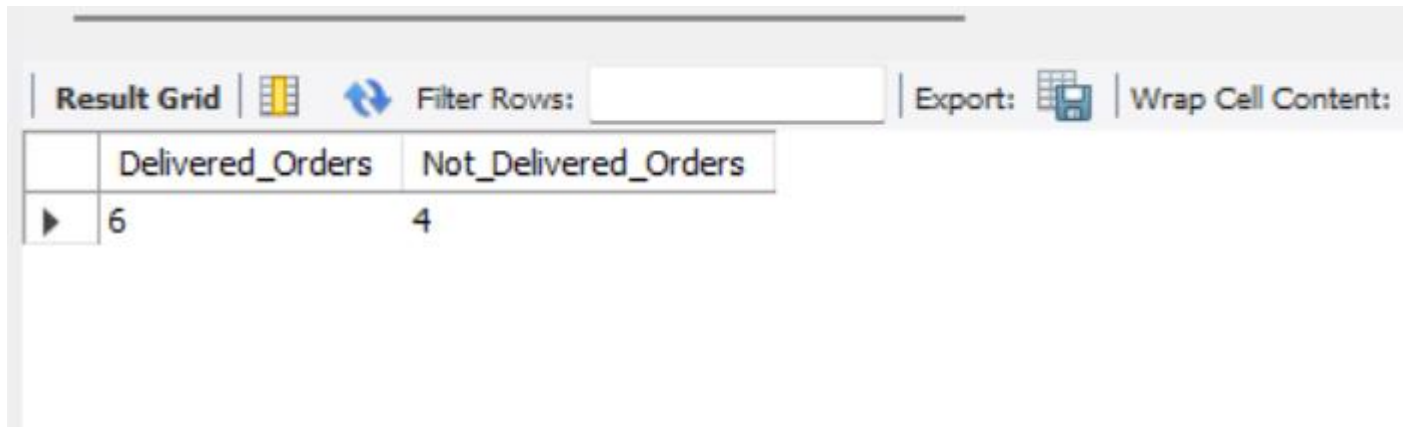
This query categorizes products into Budget, Mid-Range, and Premium based on their unit price, making product segmentation easier.

Q25. Count Delivered vs Not Delivered orders using CASE.

SQL Query

```
SELECT
SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS Delivered_Orders,
SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS
Not_Delivered_Orders
FROM orders;
```

Output



The screenshot shows a database interface with a 'Result Grid' tab. The grid has two columns: 'Delivered_Orders' and 'Not_Delivered_Orders'. The first row shows the counts 6 and 4 respectively. The interface also includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox.

	Delivered_Orders	Not_Delivered_Orders
▶	6	4

Brief Insight

This query counts delivered and non-delivered orders in a single result, providing a quick overview of order status distribution.

Q26. Explain the ACID Properties.

Answer

A – Atomicity

A transaction is completed entirely or not executed at all.

C – Consistency

The database remains valid before and after the transaction.

I – Isolation

Multiple transactions execute independently without interfering with each other.

D – Durability

Once a transaction is committed, the changes are permanently saved even if the system crashes.

Example (Bank Transfer)

- **Atomicity:** Money is either deducted and credited together, or not at all.
 - **Consistency:** The total balance remains correct after the transaction.
 - **Isolation:** Simultaneous transactions do not interfere with one another.
 - **Durability:** Once the transfer is committed, it remains saved even after a system failure.
-

Q27. Execute a SQL Transaction.

SQL Query

```
START TRANSACTION;

INSERT INTO orders
(order_id, customer_id, order_date, status, total_amount)
VALUES
(1011, 102, CURDATE(), 'Pending', 1598.00);

INSERT INTO order_items
(item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES
(5016, 1011, 206, 1, 1299.00, 0);

INSERT INTO order_items
(item_id, order_id, product_id, quantity, unit_price, discount_pct)
VALUES
(5017, 1011, 208, 1, 599.00, 0);

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 206;

UPDATE products
SET stock_qty = stock_qty - 1
WHERE product_id = 208;

COMMIT;
```

Output

✓	51	00:40:06	INSERT INTO orders (order_id, customer_id, order_date, status, total_amo
✓	52	00:40:06	INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_pric
✓	53	00:40:06	INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_pric
✓	54	00:40:06	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 2
✓	55	00:40:06	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 2
✓	56	00:40:06	COMMIT
✓	57	00:41:47	SELECT * FROM orders WHERE order_id = 1011 LIMIT 0, 1000

Brief Insight

The transaction successfully inserted a new order, added the related order items, updated product stock quantities, and committed all changes together, ensuring data consistency.

Conclusion

The ShopEase database was successfully analyzed using SQL. Various SQL operations including data retrieval, filtering, aggregation, joins, constraints, indexes, CASE statements, and transactions were implemented to answer business questions and generate meaningful insights. This assignment strengthened practical SQL skills and demonstrated how relational databases support business decision-making.